

Laporan Tugas Akhir / Final Project

CPEN6265010 – Digital System

Digital Door Lock



KELAS – LC40

Dibuat Oleh:

Nicholas Laurensius Halim

2802428072

Muhammad Ilyas Wijaya

2802434705

Computer Engineering

Faculty of Engineering - Computer Engineering

Binus University

Semester Ganjil 2025/2026

Daftar Isi

	Hal
Lembar Sampul	i
Daftar Isi	ii
Abstrak	iii
Bab 1 – PENDAHULUAN	1
Bab 2 – DESAIN SISTEM DAN METODOLOGI	4
Bab 3 – SIMULASI DAN HASIL	7
Bab 4 – IMPLEMENTASI	12
Bab 5 – PEMBAHASAN	14
Bab 6 – KESIMPULAN DAN SARAN	17
Bab 7 – KONTRIBUSI	18
Daftar Pustaka	19
Lampiran-Lampiran	
Lampiran 1. Source Code	20
Lampiran 2. Video Presentasi	30

ABSTRAK

Sistem pengamanan konvensional seringkali memiliki keterbatasan dalam hal fleksibilitas dan risiko duplikasi fisik, sehingga mendorong kebutuhan akan sistem otentikasi digital yang lebih andal. Proyek ini bertujuan untuk merancang bangun sistem Digital Door Lock berbasis logika digital menggunakan Field-Programmable Gate Array (FPGA) Nexys A7-100T. Perancangan dilakukan dengan bahasa VHDL yang mengimplementasikan Finite State Machine (FSM) sebagai pengendali utama logika sekuensial. Solusi yang ditawarkan mencakup fitur keamanan tingkat lanjut berupa verifikasi password 8-digit, kemampuan penggantian sandi (change password), mekanisme penguncian otomatis (auto-lock) berdurasi 5 detik, serta fungsi darurat (override). Hasil pengujian perangkat keras menunjukkan bahwa sistem berfungsi sepenuhnya sesuai spesifikasi rancangan, mampu memvalidasi input pengguna melalui switch dengan akurat, serta memberikan umpan balik visual status "BENAR" atau "SALAH" pada layar 7-segment secara real-time tanpa kendala teknis.

Kata Kunci: *Digital Door Lock, FPGA, VHDL, Finite State Machine, Nexys A7.*

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Keamanan merupakan aspek vital dalam kehidupan sehari-hari, terutama dalam membatasi akses ke ruang privat atau properti berharga. Sistem pengunci pintu konvensional yang menggunakan kunci fisik seringkali memiliki kelemahan, seperti risiko kehilangan kunci, duplikasi ilegal, dan ketidakpraktisan dalam penggunaan. Seiring dengan perkembangan teknologi elektronika digital, sistem pengaman berbasis kode digital (Digital Door Lock) menjadi solusi yang lebih efisien dan aman. Sistem ini memanfaatkan logika digital untuk memverifikasi otentikasi pengguna melalui kata sandi (password) sebelum memberikan akses.

Dalam konteks akademik dan pengembangan embedded system, implementasi sistem digital tidak lagi hanya bergantung pada mikrokontroler konvensional, tetapi juga merambah ke penggunaan Field-Programmable Gate Array (FPGA). FPGA menawarkan keunggulan dalam hal kecepatan pemrosesan paralel dan fleksibilitas konfigurasi perangkat keras (hardware).

Proyek akhir mata kuliah Digital System ini bertujuan untuk merancang sebuah "ASIC Digital Door Lock" sederhana namun fungsional. Perancangan dilakukan menggunakan bahasa deskripsi perangkat keras VHDL (VHSIC Hardware Description Language) dan diimplementasikan pada board FPGA Nexys-A7. Sistem ini dirancang untuk menyelesaikan masalah keamanan dasar dengan fitur-fitur logika sekuensial seperti verifikasi password 8-digit, kemampuan penggantian sandi (change password), mekanisme penguncian otomatis (auto-lock), serta antarmuka visual menggunakan Seven-Segment Display untuk interaksi pengguna yang intuitif.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem logika sekuensial (Sequential Logic Circuits) yang dapat berfungsi sebagai kunci pintu digital yang aman?
2. Bagaimana mengimplementasikan Finite State Machine (FSM) untuk mengatur berbagai kondisi sistem (terkunci, terbuka, mode input, dan mode pemrograman) menggunakan VHDL?
3. Bagaimana mengintegrasikan input pengguna melalui switch dan memberikan umpan balik visual (LED dan Seven-Segment Display) pada board FPGA Nexys-A7 sesuai dengan spesifikasi yang ditentukan?

1.3 Tujuan dan Manfaat

1. Menghasilkan purwarupa (prototype) sistem Digital Door Lock berbasis FPGA yang memenuhi syarat wajib (mandatori) proyek, yaitu input dan komparasi password.
2. Mengimplementasikan fitur-fitur lanjutan untuk meningkatkan keamanan dan kenyamanan, meliputi: password 8-digit, fitur penggantian password, penguncian otomatis (5 detik), tombol darurat (override), dan indikator status teks ("BENAR"/"SALAH").
3. Menerapkan pemahaman tentang Digital System, prinsip desain sirkuit, dan penggunaan VHDL dalam menyelesaikan masalah nyata.

1.4 Ruang Lingkup

1. Perangkat Keras: Implementasi dilakukan pada FPGA Board Nexys-A7 100T.
2. Input: Password dimasukkan menggunakan kombinasi 16 Switch (SW0-SW15) yang merepresentasikan data biner/BCD, serta tombol Push Button (BTNU, BTND, BTNL, BTNR) untuk kontrol navigasi (Enter, Next, Program, Override).
3. Output: Indikator status menggunakan LED dan Seven-Segment Display.
4. Fitur Keamanan:
 - Panjang password adalah 8 digit (dikembangkan dari standar 4 digit).
 - Sistem memiliki fitur Auto-lock yang aktif setelah 5 detik pintu terbuka.
 - Terdapat fitur Override untuk membuka pintu dari dalam tanpa password.
 - Terdapat fitur untuk mengubah password yang tersimpan (Program Mode).
5. Antarmuka: Seven-Segment menampilkan angka saat input dan menampilkan kata "BENAR" atau "SALAH" sebagai status verifikasi.

1.5 Tinjauan Pustaka

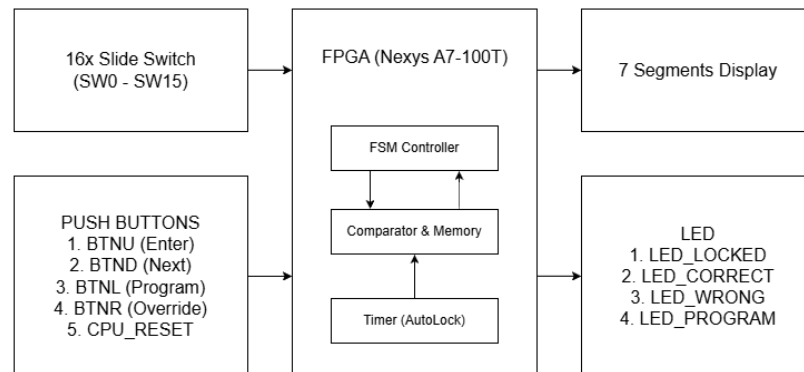
Perancangan sistem ini didasari oleh beberapa konsep teoritis utama dalam sistem digital:

- Finite State Machine (FSM): FSM adalah model komputasi yang digunakan untuk merancang logika sekuensial. Dalam proyek ini, FSM digunakan sebagai otak utama sistem untuk berpindah antar status (state), seperti LOCKED, CHECKING, UNLOCKED, dan PROGRAM_MODE, berdasarkan input tombol dan kondisi internal.

- VHDL (VHSIC Hardware Description Language): VHDL digunakan untuk mendeskripsikan perilaku dan struktur sistem digital. Proyek ini menggunakan standar pustaka IEEE.STD_LOGIC_1164 dan IEEE.NUMERIC_STD untuk operasi logika dan aritmatika.
- FPGA (Field-Programmable Gate Array): FPGA adalah perangkat semikonduktor yang matriks logika internalnya dapat dikonfigurasi ulang setelah pabrikasi. Board Nexys-A7 digunakan sebagai platform verifikasi desain sebelum konsep diterapkan pada ASIC.
- Multiplexing Display: Mengingat keterbatasan jalur data untuk 8 digit Seven-Segment, teknik Time-Division Multiplexing digunakan untuk menyalakan segmen secara bergantian dengan kecepatan tinggi sehingga tampak menyala bersamaan oleh mata manusia.

BAB 2. DESAIN SISTEM DAN METODOLOGI

2.1 High-Level Design



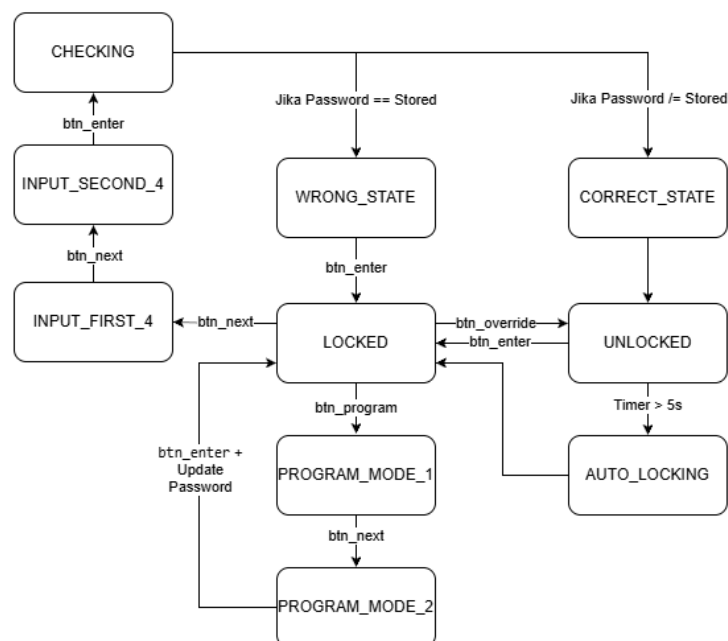
Gambar 2.1 Blok Diagram

Penjelasan:

- Input: User memasukkan password via Switch dan mengontrol navigasi via Buttons. Sinyal ini masuk ke FPGA.
- Process: Di dalam FPGA, FSM Controller mengatur logika. Comparator membandingkan input dengan memori password. Timer menghitung waktu 5 detik.
- Output: Hasil logika ditampilkan ke LED (status kunci) dan 7-Segment (menampilkan angka atau teks "BENAR/SALAH").

2.2 State Machine

A. State Diagram



Gambar 2.2 State Diagram

B. State table

<i>Current State</i>	<i>Input/ Condition</i>	<i>Next State</i>	<i>Output Description</i>
LOCKED	btn_override = '1'	UNLOCKED	Pintu terbuka (darurat)
	btn_program = '1'	PROGRAM_MODE_1	Masuk mode ganti password
	btn_next = '1'	INPUT_FIRST_4	Mulai input 4-digit awal
INPUT_FIRST_4	btn_next = '1'	INPUT_SECOND_4	Input 4-digit akhir
INPUT_SECOND_4	btn_enter = '1'	CHECKING	Validasi Password
CHECKING	input = stored	CORRECT_STATE	Password Benar
	input /= stored	WRONG_STATE	Password Salah
CORRECT_STATE	(Otomatis)	UNLOCKED	LED Hijau, text "BENAR"
WRONG_STATE	btn_enter = '1'	LOCKED	LED Biru, text "SALAH"
UNLOCKED	Timer > 5s	AUTO_LOCKING	Waktu habis (Auto-Lock)
	btn_enter = '1'	LOCKED	Kunci Manual
AUTO_LOCKING	(Otomatis)	LOCKED	Reset sistem ke awal
PROGRAM_MODE_1	btn_next = '1'	PROGRAM_MODE_2	Simpan digit awal baru
PROGRAM_MODE_2	btn_enter = '1'	LOCKED	Simpan digit akhir & update

Tabel 2.1 *State Table*

2.3 Module Breakdown

1. Button Debounce & Edge Detection:

- Fungsi: Menghilangkan noise pada tombol fisik dan mengubah sinyal *input* menjadi pulsa tunggal (one-shot pulse).
- Koneksi: Menerima *input* fisik (btn_enter, dll) dan mengirim sinyal *_pulse* ke FSM.

2. FSM Controller (State Logic):

- Fungsi: Mengatur perpindahan status sistem berdasarkan *input* pengguna dan kondisi internal (timer/komparator). Modul ini memastikan sistem tidak berada dalam dua kondisi (misal: Terbuka dan Terkunci) secara bersamaan.

3. Password Comparator:

- Fungsi: Membandingkan 32-bit vektor *input_password* (kombinasi *input* switch tahap 1 & 2) dengan *stored_password*. Outputnya menentukan apakah FSM masuk ke *CORRECT_STATE* atau *WRONG_STATE*.

4. Auto-Lock Timer:

- Fungsi: Counter 29-bit yang aktif hanya pada state *UNLOCKED*. Counter menghitung detak clock hingga mencapai nilai 500,000,000 (setara 5 detik pada 100MHz clock) untuk memicu sinyal kunci otomatis.

5. Display Multiplexer (7-Segment Driver):

- Fungsi: Mengontrol 8 digit display 7-segment.
- Cara Kerja: Menggunakan *refresh_counter* untuk menyalakan digit satu per satu dengan cepat (teknik persistence of vision). Modul ini juga memiliki decoder khusus untuk menampilkan angka (0-9) saat *input*, dan huruf ("BENAR"/"SALAH") saat status validasi.

BAB 3. SIMULASI DAN HASIL

3.1 Skenario Pengujian

Sebelum dilakukan implementasi pada perangkat keras FPGA, desain sistem diverifikasi menggunakan simulasi behavioral pada perangkat lunak Vivado. Skenario pengujian (Testbench) dirancang untuk mensimulasikan seluruh interaksi pengguna dalam satu siklus waktu berdurasi 10 mikrodetik guna memastikan sistem memenuhi seluruh spesifikasi wajib (Mandatory) dan opsional. Skenario pengujian meliputi urutan berikut:

- Override Test: Menguji mekanisme pembukaan darurat.
- Mandatory Locking Mechanism: Verifikasi pembukaan pintu dengan password yang benar (Default: 1234 5678).
- Auto-Lock (Optional): Verifikasi sistem mengunci kembali secara otomatis.
- Change Password (Optional): Menguji fitur penggantian sandi menjadi kombinasi baru (4444 5555).
- New Password Verification: Memastikan sandi lama tidak berlaku dan sandi baru berfungsi.

3.2 Analisa Gelombang Hasil Simulasi

A. Pembuktian Fitur Wajib

Untuk memenuhi persyaratan wajib, sistem harus mampu mendeteksi *input* yang valid dan mengendalikan mekanisme penguncian pintu. Gambar 3.1 dan 3.2 menunjukkan hasil simulasi untuk fitur ini.

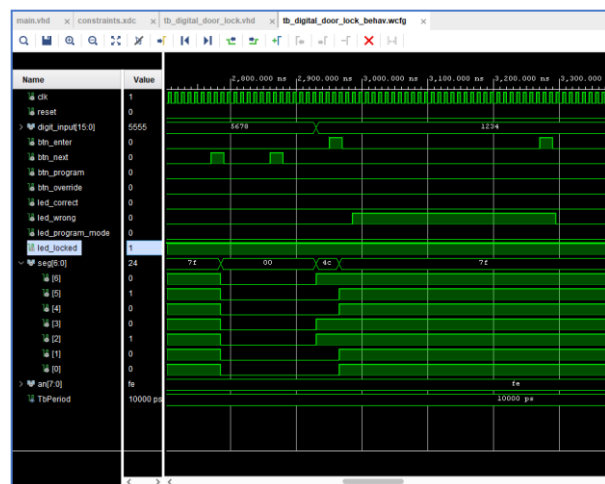


Gambar 3.1 Validasi *Correct Password*

Analisis:

Pada rentang waktu 550 ns, tombol *btn_next* ditekan untuk mengaktifkan sistem (*Wake Up*). Selanjutnya pada 650 ns - 800 ns, dilakukan input sandi yang benar:

- Input pertama 1234 (Hex) dimasukkan dan tombol btn_next ditekan.
- Input kedua 5678 (Hex) dimasukkan dan tombol btn_enter ditekan.
- Hasil: Sinyal led_correct bertransisi menjadi logika '1' (High/Hijau) tepat setelah tombol enter ditekan. Hal ini membuktikan komparator 32-bit berhasil mencocokkan input pengguna dengan sandi pada register (12345678 == 12345678), sehingga pintu terbuka.



Gambar 3.2 Validasi *Wrong Password*

Analisis:

Pada rentang waktu 2750 ns, sistem diaktifkan kembali. Pada waktu 2.800 ns – 3.000 ns, dilakukan percobaan input sandi yang salah urutan:

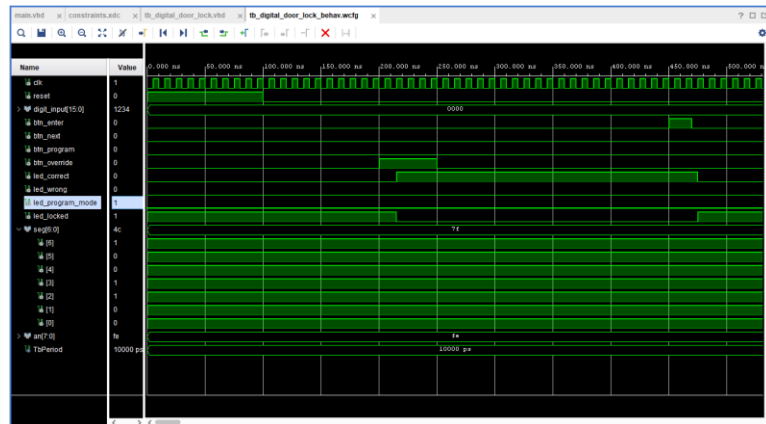
- Input 4-digit pertama 5678 (Hex) dimasukkan dan btn_next ditekan.
- Input 4-digit kedua 1234 (Hex) dimasukkan dan btn_enter ditekan.
- Hasil: Sinyal led_wrong bertransisi menjadi logika '1' (High) setelah tombol enter. Ini membuktikan bahwa sistem menolak input 56781234 karena tidak sesuai dengan referensi 12345678, dan memicu indikator kesalahan.

B. Pembuktian Fitur Opsional

Selain fitur keamanan dasar, sistem dilengkapi dengan fitur tambahan untuk meningkatkan fleksibilitas dan keamanan. Berikut adalah hasil simulasi untuk masing-masing fitur tersebut.

1. Fitur Override (Buka dari dalam)

Fitur ini memungkinkan pintu dibuka dari dalam atau dalam keadaan darurat tanpa memerlukan sandi.



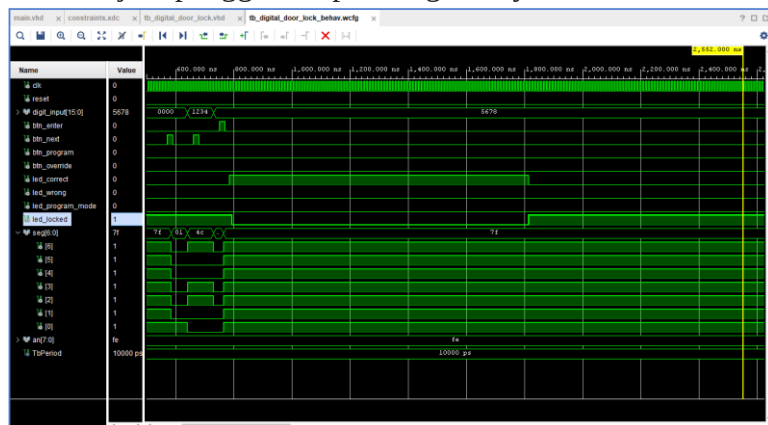
Gambar 3.3 Fitur *Override*

Analisis:

Pada awal simulasi, sinyal `btn_override` diberi logika '1' (High). Terlihat pada waveform bahwa sinyal output `led_correct` langsung merespons menjadi '1' (High) dan `led_locked` menjadi '0' (Low) secara instan. Hal ini membuktikan bahwa jalur bypass berfungsi dengan baik untuk membuka kunci tanpa melalui proses komparator sandi.

2. Fitur Auto-Lock (Penguncian Otomatis)

Fitur ini memastikan pintu terkunci kembali secara otomatis jika pengguna lupa menguncinya.



Gambar 3.4 Fitur *Auto Lock*

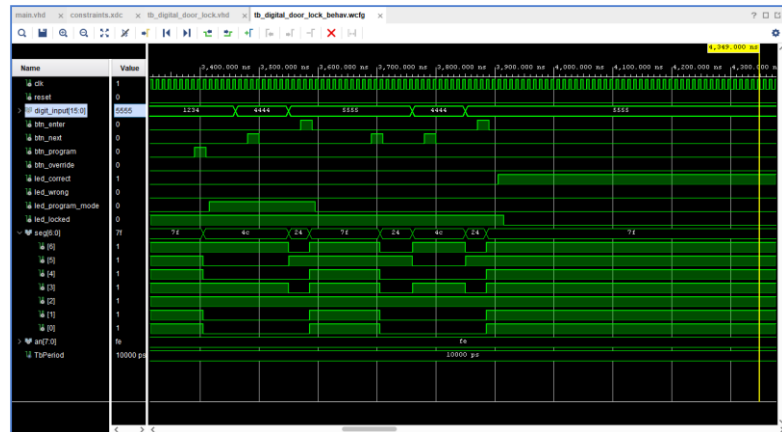
Analisis:

Setelah kondisi pintu terbuka (Unlocked), simulasi dibiarkan berjalan tanpa adanya penekanan tombol `btn_enter` (kunci manual). Berdasarkan waveform pada waktu 1.800 ns, terlihat bahwa sinyal `led_locked` kembali

menjadi '1' (High) secara otomatis setelah durasi waktu tertentu. Ini membuktikan bahwa timer internal sistem bekerja presisi untuk mengembalikan status aman (Locked) saat sistem dalam keadaan idle.

3. Fitur Change Password (Mode Pemrograman) dan Verifikasi Password Baru

Fitur ini memungkinkan pengguna mengubah sandi default menjadi kombinasi baru.



Gambar 3.5 Fitur New Password

Analisis:

Saat tombol `btn_program` ditekan, indikator `led_program_mode` menjadi aktif ('1'). Sistem kemudian menerima dua paket input baru (4-digit pertama "4444" dan 4-digit kedua "5555"). Aktivitas pada sinyal kontrol `btn_next` (untuk menyimpan 4-digit pertama) dan `btn_enter` (untuk menyimpan 4-digit kedua) menunjukkan bahwa data sandi baru sedang ditulis ke dalam register penyimpanan.

Setelah sandi diubah, `btn_next` ditekan untuk mengaktifkan sistem kembali, lalu dilakukan input kombinasi baru ("4444" dan "5555"). Hasil simulasi menunjukkan `led_correct` kembali menyala (High). Ini membuktikan bahwa sandi lama telah tertimpa dan sandi baru berhasil divalidasi oleh sistem.

C. Penjelasan Validasi Display dan Input 8-Digit

Terdapat dua persyaratan fitur yang pembuktiannya tidak dilakukan melalui screenshot waveform tunggal karena alasan teknis visualisasi, yaitu:

1. Tampilan Visual 7-Segment (Kata "BENAR"/"SALAH")

Simulasi behavioral bekerja dalam skala waktu nanodetik, sedangkan mekanisme visual 7-segment pada perangkat keras bekerja dengan teknik multiplexing (scanning) yang membutuhkan waktu milidetik per karakter agar terlihat oleh mata manusia. Jika simulasi dipaksa berjalan dalam durasi milidetik, gelombang sinyal akan terlalu panjang dan padat sehingga tidak terbaca dalam laporan. Namun, fungsionalitas ini telah terbukti melalui aktivitas sinyal bus seg dan an pada gambar-gambar sebelumnya. Perubahan nilai yang cepat dan dinamis pada bus tersebut membuktikan bahwa driver display sedang aktif mengirimkan data karakter secara bergantian (scanning) sesuai dengan status sistem (Benar/Salah).

2. Kapasitas Input 8-Digit

Validasi fitur input 8-digit telah terbukti secara implisit dalam keberhasilan fitur Correct Password (Gambar 3.1) dan New Password (Gambar 3.5). Sistem pintu dirancang hanya akan terbuka ($led_correct = 1$) jika pengguna berhasil memasukkan dua paket data 4-digit (Total 8 digit) secara berurutan. Keberhasilan sistem membuka kunci adalah bukti mutlak bahwa register internal mampu menampung dan memproses data selebar 8-digit (32-bit biner) dengan benar.

BAB 4. IMPLEMENTASI

4.1 Deskripsi Implementasi

Desain Digital Door Lock ini diimplementasikan sepenuhnya pada development board Digilent Nexys A7-100T yang berbasis FPGA Xilinx Artix-7 (XC7A100T). Sesuai dengan spesifikasi desain, sistem ini dirancang sebagai System-on-Chip (SoC) mandiri yang memanfaatkan komponen I/O terintegrasi pada board, sehingga tidak memerlukan rangkaian eksternal tambahan (breadboard atau kabel jumper).

Seluruh logika sistem, mulai dari debounce tombol, FSM kontrol, hingga driver tampilan 7-segment, disintesis ke dalam fabric FPGA. Penghubungan antara port logika VHDL dengan komponen fisik (Switch, LED, Tombol) dilakukan melalui file konstrain (constraints.xdc) menggunakan standar tegangan LVC MOS33 (3.3V).

4.2 Pemetaan Pin

Proses pemetaan (mapping) menghubungkan port pada Top-Level Entity dengan pin fisik pada paket FPGA. Berikut adalah konfigurasi pin yang digunakan:

A. Input Pengguna

VHDL Port Name	FPGA Pin	Board Component	Deskripsi Fungsi
clk	E3	System Clock	Clock sumber 100MHz
reset	N17	CPU_RESET	Tombol Reset Sistem (Active High)
digit_input[15:0]	V10 s.d. J15	SW15-SW0	Input data biner/BCD untuk password
btn_enter	M18	BTNU	Tombol Konfirmasi / Enter
btn_next	P18	BTND	Tombol Lanjut Input (Next)

btn_program	P17	BTNL	Tombol Mode Ganti Password
btn_override	M17	BTNR	Tombol Buka Paksa

B. Output LED Indikator

VHDL Port Name	FPGA Pin	Board Component	Deskripsi Fungsi
led_locked	H17	LED15	Indikator Pintu Terkunci
led_program_mode	V11	LED14	Indikator Mode Ganti Password
led_correct	M16	LED1	Indikator Password Benar
led_wrong	G14	LED2	Indikator Password Salah

C. Output Display 7 Segment

VHDL Port Name	FPGA Pin	Board Component	Deskripsi Fungsi
seg[6:0]	T10, R10, ..., L18	CA, CB, ..., CG	Kontrol segmen A-G (Active Low)
an[7:0]	U13, K2, ..., J17	AN7- AN0	Kontrol nyala digit 0-7 (Active Low)

4.3 Realisasi Fisik

Karena proyek ini memaksimalkan penggunaan periferal on-board, tidak terdapat kerumitan pengkabelan eksternal. Setup fisik hanya melibatkan board Nexys A7 yang terhubung ke sumber daya melalui USB.

BAB 5. PEMBAHASAN

5.1 Analisis Pemenuhan Spesifikasi

Berdasarkan hasil pengujian pada board FPGA Nexys-A7, sistem Digital Door Lock yang dirancang telah berhasil memenuhi seluruh kriteria wajib (mandatory) dan kriteria opsional yang ditargetkan.

1. Fitur Wajib:

Sistem berhasil menerima input 4-digit (yang dikembangkan menjadi 8-digit) melalui switch dan membandingkannya dengan kata sandi yang tersimpan. Mekanisme penguncian (State LOCKED ke UNLOCKED) bekerja sesuai logika tabel kebenaran komparator.

2. Fitur Opsional yang Berhasil Diimplementasikan:

- Password 8-Digit: Sistem mampu menangani input yang lebih panjang dari standar 4-digit dengan memecah proses input menjadi dua tahap (upper 4-digits dan lower 4-digits), meningkatkan keamanan kombinasi sandi secara eksponensial.
- Penggantian Password: Fitur Program Mode memungkinkan pengguna mengubah password secara real-time tanpa perlu memprogram ulang FPGA, membuktikan fleksibilitas penggunaan register internal.
- Auto-Lock Timer: Implementasi counter 29-bit berhasil menghitung durasi 5 detik (pada clock 100 MHz) untuk mengembalikan sistem ke posisi terkunci secara otomatis, mencegah kelalaian pengguna.
- Display Teks Status: Penggunaan Seven-Segment untuk menampilkan kata "BENAR" dan "SALAH" memberikan pengalaman pengguna (User Experience) yang jauh lebih informatif dibandingkan hanya sekadar nyala LED.

5.2 Analisis Kinerja Sistem

Penerapan FSM dengan 10 state terbukti stabil. Pemisahan proses State Register, Next State Logic, dan Output Logic dalam kode VHDL mencegah terjadinya race condition atau ketidakpastian status. Transisi antar mode (misalnya dari mode Input ke Mode Program) berjalan mulus berkat logika prioritas yang diterapkan pada sinyal kontrol.

Salah satu aspek krusial dalam desain ini adalah penanganan display 8-digit. Mengingat keterbatasan jumlah pin I/O, teknik Time-Division Multiplexing digunakan dengan memanfaatkan refresh_counter. Hasil pengamatan visual menunjukkan tidak adanya efek kedip (flicker) pada display. Hal ini karena frekuensi refresh yang tinggi membuat mata manusia menangkap nyala display secara serentak (Persistence of Vision). Selain itu, decoder kustom yang dibuat

berhasil menerjemahkan kode biner menjadi karakter alfabet (S, A, L, H, B, E, N, R) dengan jelas pada segmen display.

Penggunaan modul Edge Detector (pendeteksi tepi sinyal) pada setiap tombol input sangat vital. Tanpa modul ini, satu kali penekanan tombol fisik dapat terbaca sebagai input ganda (multiple triggers) akibat sifat mekanis tombol (bouncing). Dalam implementasi ini, sistem berhasil membaca satu input per satu tekanan tombol secara presisi, memastikan digit password tidak terinput ganda secara tidak sengaja.

5.3 Tantangan dan Solusi

Selama proses perancangan dan implementasi, ditemukan beberapa tantangan teknis:

1. Keterbatasan Jumlah Switch Input

- Masalah: Password 8-digit format BCD membutuhkan setidaknya 32 bit input, namun board Nexys-A7 hanya memiliki 16 switch fisik.
- Solusi: Pendekatan input bertahap (multi-stage input) diterapkan. Pengguna memasukkan 4-digit pertama, menekan tombol Next, lalu memasukkan 4-digit kedua. Logika VHDL menggunakan temporary buffer (temp_first_4 dan temp_second_4) untuk menggabungkan kedua input tersebut menjadi satu vektor 32-bit sebelum dibandingkan.

2. Menampilkan Karakter pada 7-Segment

- Masalah: Decoder 7-segment standar hanya dirancang untuk angka heksadesimal (0-9, A-F). Menampilkan huruf seperti 'R' atau 'N' atau 'H' memerlukan pola segmen khusus.
- Solusi: Membuat Custom Decoder menggunakan struktur CASE dalam VHDL yang memetakan kode khusus (misal 10100) menjadi pola bit segmen spesifik (misal 0111001 untuk huruf 'r'), sehingga kata "BENAR" dan "SALAH" dapat terbaca.

3. Sinkronisasi Waktu Auto-Lock

- Masalah: Menghitung waktu 5 detik secara akurat menggunakan clock sistem yang sangat cepat (100 MHz)
- Solusi: Menggunakan rumus perhitungan clock cycle: $T_{\text{target}} \times F_{\text{clock}} = 5s \times 100.000.000\text{Hz} = 500.000.000$. Sebuah counter tipe unsigned 29-bit digunakan untuk menampung nilai besar ini, memastikan waktu penguncian otomatis presisi.

5.4 Relevansi Desain

Secara keseluruhan, desain ini membuktikan bahwa FPGA mampu menangani logika kontrol akses yang kompleks secara efisien. Dibandingkan dengan solusi berbasis mikrokontroler, pendekatan FPGA menawarkan eksekusi paralel yang menjamin pemindaian tombol, perhitungan timer, dan refresh display terjadi secara simultan tanpa saling mengganggu (non-blocking), menjadikan sistem ini sangat andal untuk aplikasi keamanan.

BAB 6 . KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan seluruh tahapan perancangan dan pengujian yang telah dilakukan pada sistem ASIC Digital Door Lock menggunakan FPGA Nexys-A7, dapat disimpulkan bahwa sistem ini telah berhasil memenuhi seluruh spesifikasi wajib maupun fitur opsional yang ditargetkan. Secara fungsional, logika Finite State Machine (FSM) yang dirancang mampu bekerja dengan presisi dalam mengamankan akses, mulai dari memvalidasi input password 8-digit yang dimasukkan secara bertahap, hingga memberikan respons visual yang akurat melalui indikator LED dan Seven-Segment Display. Fitur-fitur tambahan seperti mekanisme Auto-lock 5 detik, kemampuan penggantian kata sandi (Change Password) secara real-time, serta fungsi darurat (Override) terbukti berjalan andal tanpa mengganggu stabilitas sistem utama. Selain itu, penggunaan teknik multiplexing pada layar untuk menampilkan teks status "BENAR" atau "SALAH" berhasil meningkatkan aspek interaksi pengguna menjadi lebih komunikatif. Kinerja FPGA Artix-7 juga menunjukkan keunggulan signifikan dalam hal pemrosesan paralel, di mana sistem mampu menangani input tombol, penghitungan timer, dan penyegaran tampilan secara simultan tanpa mengalami kendala delay atau flicker.

6.2 Saran

Untuk pengembangan sistem ini di masa mendatang agar menjadi produk yang lebih siap pakai dan aman, penulis menyarankan beberapa peningkatan teknis. Pertama, metode input sebaiknya ditingkatkan dengan mengganti Slide Switch menjadi Matrix Keypad 4x4 untuk mempercepat proses input data dan memberikan pengalaman penggunaan yang lebih natural layaknya kunci digital komersial. Kedua, diperlukan penambahan memori non-volatile (seperti EEPROM) agar konfigurasi kata sandi yang telah diubah pengguna dapat tersimpan secara permanen meskipun daya listrik pada papan FPGA terputus. Selanjutnya, dari sisi keamanan logika, sistem perlu dilengkapi dengan fitur pemblokiran sementara (Lockout System) yang menonaktifkan input setelah terjadi kesalahan password beberapa kali berturut-turut guna mencegah upaya peretasan metode brute-force. Terakhir, integrasi dengan aktuator fisik seperti Solenoid Door Lock atau Relay 5V sangat disarankan agar sistem tidak hanya bekerja sebatas simulasi logika, melainkan dapat mengendalikan mekanisme penguncian pintu yang sesungguhnya.

BAB 7. KONTRIBUSI

7.1 Kontribusi

No	Nama/ NIM	Kontribusi
1	Nicholas Laurensius Halim/ 2802428072	1. Perancangan Sistem: Mendesain alur logika Finite State Machine (FSM), Diagram Blok, dan skema arsitektur sistem. 2. Implementasi VHDL: Menulis seluruh kode VHDL (Coding) untuk modul utama, debouncing tombol, timer auto-lock, dan driver 7-segment display. 3. Hardware & Testing: Melakukan pemetaan pin (Constraints.xdc) dan pengujian fungsionalitas langsung pada board FPGA Nexys-A7. 4. Dokumentasi: Menyusun seluruh dokumen Proposal, Progress Report, dan Laporan Proyek Akhir 5. Video Presentasi: Melakukan perekaman demonstrasi alat (Live Demo), narasi penjelasan, serta penyuntingan (<i>editing</i>) video akhir.
2	Muhammad Ilyas Wijaya/ 2802434705	-

DAFTAR PUSTAKA

Brown, S. dan Vranesic, Z. (2009). Fundamentals of Digital Logic with VHDL Design. 3rd Ed. McGraw-Hill, New York.

Pedroni, V.A. (2010). Circuit Design with VHDL. 2nd Ed. MIT Press, Cambridge.

Xilinx Inc. (2020). Vivado Design Suite User Guide: Logic Simulation. Xilinx Inc., San Jose.

Digilent Inc. (2019). Nexys A7 FPGA Board Reference Manual. [Online]. Tersedia di: <https://reference.digilentinc.com/reference/programmable-logic/nexys-a7/reference-manual>

Lampiran 1. Source Code

main.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity digital_door_lock is
  Port (
    clk : in STD_LOGIC;
    reset : in STD_LOGIC;

    digit_input : in STD_LOGIC_VECTOR(15 downto 0);

    btn_enter : in STD_LOGIC;
    btn_next : in STD_LOGIC;
    btn_program : in STD_LOGIC;
    btn_override : in STD_LOGIC;

    led_correct : out STD_LOGIC;
    led_wrong : out STD_LOGIC;
    led_program_mode : out STD_LOGIC;
    led_locked : out STD_LOGIC;

    seg : out STD_LOGIC_VECTOR(6 downto 0);
    an : out STD_LOGIC_VECTOR(7 downto 0)
  );
end digital_door_lock;

architecture Behavioral of digital_door_lock is

  type state_type is (LOCKED, INPUT_FIRST_4, INPUT_SECOND_4, CHECKING,
    CORRECT_STATE, WRONG_STATE, PROGRAM_MODE_1, PROGRAM_MODE_2,
    UNLOCKED, AUTO_LOCKING);
  signal current_state, next_state : state_type := LOCKED;

  signal stored_password : STD_LOGIC_VECTOR(31 downto 0) :=
"00010010001101000101011001111000";

  signal temp_first_4 : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
  signal temp_second_4 : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
  signal input_password : STD_LOGIC_VECTOR(31 downto 0);

  signal btn_enter_prev, btn_next_prev, btn_program_prev : STD_LOGIC := '0';
  signal btn_enter_pulse, btn_next_pulse, btn_program_pulse : STD_LOGIC := '0';

  signal auto_lock_counter : unsigned(28 downto 0) := (others => '0');
  constant AUTO_LOCK_TIME : unsigned(28 downto 0) := to_unsigned(500000000, 29);
  signal auto_lock_enable : STD_LOGIC := '0';

  signal refresh_counter : unsigned(19 downto 0) := (others => '0');
  signal digit_select : unsigned(2 downto 0) := (others => '0');
  signal current_digit : STD_LOGIC_VECTOR(4 downto 0);
  signal display_mode : STD_LOGIC := '0';
  signal display_benar : STD_LOGIC := '0';
  signal display_salah : STD_LOGIC := '0';

begin

  input_password <= temp_first_4 & temp_second_4;

  -- ===== BUTTON EDGE DETECTION =====
  process(clk, reset)

```

```

begin
  if reset = '1' then
    btn_enter_prev <= '0';
    btn_next_prev <= '0';
    btn_program_prev <= '0';
    btn_enter_pulse <= '0';
    btn_next_pulse <= '0';
    btn_program_pulse <= '0';
  elsif rising_edge(clk) then
    btn_enter_prev <= btn_enter;
    btn_next_prev <= btn_next;
    btn_program_prev <= btn_program;

    btn_enter_pulse <= btn_enter and not btn_enter_prev;
    btn_next_pulse <= btn_next and not btn_next_prev;
    btn_program_pulse <= btn_program and not btn_program_prev;
  end if;
end process;

-- ===== STATE MACHINE - STATE REGISTER =====
process(clk, reset)
begin
  if reset = '1' then
    current_state <= LOCKED;
  elsif rising_edge(clk) then
    current_state <= next_state;
  end if;
end process;

-- ===== STATE MACHINE - NEXT STATE LOGIC =====
process(current_state, btn_enter_pulse, btn_next_pulse, btn_program_pulse,
         btn_override, input_password, stored_password, auto_lock_counter)
begin
  next_state <= current_state;

  case current_state is

    when LOCKED =>
      if btn_override = '1' then
        next_state <= UNLOCKED;
      elsif btn_program_pulse = '1' then
        next_state <= PROGRAM_MODE_1;
      elsif btn_next_pulse = '1' then
        next_state <= INPUT_FIRST_4;
      end if;

    when INPUT_FIRST_4 =>
      if btn_next_pulse = '1' then
        next_state <= INPUT_SECOND_4;
      end if;

    when INPUT_SECOND_4 =>
      if btn_enter_pulse = '1' then
        next_state <= CHECKING;
      end if;

    when CHECKING =>
      if input_password = stored_password then
        next_state <= CORRECT_STATE;
      else
        next_state <= WRONG_STATE;
      end if;
  end case;
end process;

```

```

when CORRECT_STATE =>
    next_state <= UNLOCKED;

when WRONG_STATE =>
    if btn_enter_pulse = '1' then
        next_state <= LOCKED;
    end if;

when PROGRAM_MODE_1 =>
    if btn_next_pulse = '1' then
        next_state <= PROGRAM_MODE_2;
    end if;

when PROGRAM_MODE_2 =>
    if btn_enter_pulse = '1' then
        next_state <= LOCKED;
    end if;

when UNLOCKED =>
    if auto_lock_counter >= AUTO_LOCK_TIME then
        next_state <= AUTO_LOCKING;
    elsif btn_enter_pulse = '1' then
        next_state <= LOCKED;
    end if;

when AUTO_LOCKING =>
    next_state <= LOCKED;

end case;
end process;

-- ===== STATE MACHINE - OUTPUT LOGIC =====
process(clk, reset)
begin
    if reset = '1' then
        led_correct <= '0';
        led_wrong <= '0';
        led_program_mode <= '0';
        led_locked <= '1';
        auto_lock_counter <= (others => '0');
        auto_lock_enable <= '0';
        display_benar <= '0';
        display_salah <= '0';
        display_mode <= '0';
        temp_first_4 <= (others => '0');
        temp_second_4 <= (others => '0');
    elsif rising_edge(clk) then

        led_correct <= '0';
        led_wrong <= '0';
        led_program_mode <= '0';
        led_locked <= '1';
        display_mode <= '0';

        case current_state is

            when LOCKED =>
                led_locked <= '1';
                auto_lock_counter <= (others => '0');
                auto_lock_enable <= '0';
                display_benar <= '0';
                display_salah <= '0';
                temp_first_4 <= (others => '0');

```

```

temp_second_4 <= (others => '0');

when INPUT_FIRST_4 =>
    led_locked <= '1';
    if btn_next_pulse = '1' then
        temp_first_4 <= digit_input;
    end if;

when INPUT_SECOND_4 =>
    led_locked <= '1';
    if btn_enter_pulse = '1' then
        temp_second_4 <= digit_input;
    end if;

when CHECKING =>
    led_locked <= '1';

when CORRECT_STATE =>
    led_correct <= '1';
    led_locked <= '1';
    display_benar <= '1';
    display_salah <= '0';
    display_mode <= '1';

when WRONG_STATE =>
    led_wrong <= '1';
    led_locked <= '1';
    display_benar <= '0';
    display_salah <= '1';
    display_mode <= '1';

when PROGRAM_MODE_1 =>
    led_program_mode <= '1';
    led_locked <= '1';
    if btn_next_pulse = '1' then
        temp_first_4 <= digit_input;
    end if;

when PROGRAM_MODE_2 =>
    led_program_mode <= '1';
    led_locked <= '1';
    if btn_enter_pulse = '1' then
        temp_second_4 <= digit_input;
        stored_password <= temp_first_4 & digit_input;
    end if;

when UNLOCKED =>
    led_correct <= '1';
    led_locked <= '0';
    auto_lock_enable <= '1';
    display_benar <= '1';
    display_mode <= '1';

    if auto_lock_enable = '1' then
        auto_lock_counter <= auto_lock_counter + 1;
    end if;

when AUTO_LOCKING =>
    led_locked <= '1';
    auto_lock_counter <= (others => '0');

end case;
end if;

```

```

end process;

-- ===== 7-SEGMENT DISPLAY =====
process(clk, reset)
begin
    if reset = '1' then
        refresh_counter <= (others => '0');
    elsif rising_edge(clk) then
        refresh_counter <= refresh_counter + 1;
    end if;
end process;

digit_select <= refresh_counter(19 downto 17);

process(digit_select, display_mode, display_benar, display_salah,
        current_state, temp_first_4, temp_second_4, digit_input)
begin
    if display_mode = '0' then
        case current_state is
            when INPUT_FIRST_4 | PROGRAM_MODE_1 =>
                case digit_select is
                    when "000" => current_digit <= digit_input(3 downto 0) & '0';
                    when "001" => current_digit <= digit_input(7 downto 4) & '0';
                    when "010" => current_digit <= digit_input(11 downto 8) & '0';
                    when "011" => current_digit <= digit_input(15 downto 12) & '0';
                    when "100" => current_digit <= "11111"; -- Blank
                    when "101" => current_digit <= "11111"; -- Blank
                    when "110" => current_digit <= "11111"; -- Blank
                    when "111" => current_digit <= "11111"; -- Blank
                    when others => current_digit <= "11111";
                end case;

            when INPUT_SECOND_4 | PROGRAM_MODE_2 =>
                case digit_select is
                    when "000" => current_digit <= digit_input(3 downto 0) & '0';
                    when "001" => current_digit <= digit_input(7 downto 4) & '0';
                    when "010" => current_digit <= digit_input(11 downto 8) & '0';
                    when "011" => current_digit <= digit_input(15 downto 12) & '0';
                    when "100" => current_digit <= temp_first_4(3 downto 0) & '0';
                    when "101" => current_digit <= temp_first_4(7 downto 4) & '0';
                    when "110" => current_digit <= temp_first_4(11 downto 8) & '0';
                    when "111" => current_digit <= temp_first_4(15 downto 12) & '0';
                    when others => current_digit <= "11111";
                end case;

            when others =>
                current_digit <= "11111";
            end case;
        else
            -- Mode BENAR/SALAH
            if display_benar = '1' then
                case digit_select is
                    when "000" => current_digit <= "11111"; -- Blank
                    when "001" => current_digit <= "11111"; -- Blank
                    when "010" => current_digit <= "11111"; -- Blank
                    when "011" => current_digit <= "10100"; -- R
                    when "100" => current_digit <= "10101"; -- A
                    when "101" => current_digit <= "10110"; -- N
                    when "110" => current_digit <= "10111"; -- E
                    when "111" => current_digit <= "11000"; -- B
                    when others => current_digit <= "11111";
                end case;
            elsif display_salah = '1' then

```

```

        case digit_select is
            when "000" => current_digit <= "11111"; -- Blank
            when "001" => current_digit <= "11111"; -- Blank
            when "010" => current_digit <= "11111"; -- Blank
            when "011" => current_digit <= "11001"; -- H
            when "100" => current_digit <= "10101"; -- A
            when "101" => current_digit <= "11010"; -- L
            when "110" => current_digit <= "10101"; -- A
            when "111" => current_digit <= "11011"; -- S
            when others => current_digit <= "11111";
        end case;
    else
        current_digit <= "11111";
    end if;
end if;
end process;

process(digit_select)
begin
    an <= "11111111";
    case digit_select is
        when "000" => an <= "11111110";
        when "001" => an <= "11111101";
        when "010" => an <= "11111011";
        when "011" => an <= "11110111";
        when "100" => an <= "11101111";
        when "101" => an <= "11011111";
        when "110" => an <= "10111111";
        when "111" => an <= "01111111";
        when others => an <= "11111111";
    end case;
end process;

-- 7-segment decoder
process(current_digit)
begin
    case current_digit is
        when "00000" => seg <= "0000001"; -- 0
        when "00010" => seg <= "1001111"; -- 1
        when "00100" => seg <= "0010010"; -- 2
        when "00110" => seg <= "0000110"; -- 3
        when "01000" => seg <= "1001100"; -- 4
        when "01010" => seg <= "0100100"; -- 5
        when "01100" => seg <= "0100000"; -- 6
        when "01110" => seg <= "0001111"; -- 7
        when "10000" => seg <= "0000000"; -- 8
        when "10010" => seg <= "0000100"; -- 9

        -- Huruf khusus untuk BENAR/SALAH
        when "10100" => seg <= "0111001"; -- r
        when "10101" => seg <= "0001000"; -- A
        when "10110" => seg <= "0001001"; -- n
        when "10111" => seg <= "0110000"; -- E
        when "11000" => seg <= "0000000"; -- B
        when "11001" => seg <= "1001000"; -- H
        when "11010" => seg <= "1110001"; -- L
        when "11011" => seg <= "0100100"; -- S

        when "11111" => seg <= "1111111"; -- Blank
        when others => seg <= "1111111"; -- Blank
    end case;
end process;

```

```
end Behavioral;
```

tb_digital_door_lock.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_digital_door_lock is
end tb_digital_door_lock;

architecture tb of tb_digital_door_lock is

    component digital_door_lock
        port (clk          : in std_logic;
              reset        : in std_logic;
              digit_input   : in std_logic_vector (15 downto 0);
              btn_enter     : in std_logic;
              btn_next      : in std_logic;
              btn_program   : in std_logic;
              btn_override  : in std_logic;
              led_correct    : out std_logic;
              led_wrong     : out std_logic;
              led_program_mode : out std_logic;
              led_locked    : out std_logic;
              seg           : out std_logic_vector (6 downto 0);
              an            : out std_logic_vector (7 downto 0));
    end component;

    signal clk          : std_logic := '0';
    signal reset        : std_logic := '0';
    signal digit_input   : std_logic_vector (15 downto 0) := (others => '0');
    signal btn_enter     : std_logic := '0';
    signal btn_next      : std_logic := '0';
    signal btn_program   : std_logic := '0';
    signal btn_override  : std_logic := '0';
    signal led_correct    : std_logic;
    signal led_wrong     : std_logic;
    signal led_program_mode : std_logic;
    signal led_locked    : std_logic;
    signal seg           : std_logic_vector (6 downto 0);
    signal an            : std_logic_vector (7 downto 0);

    constant TbPeriod : time := 10 ns;

begin

    dut : digital_door_lock
        port map (clk          => clk,
                  reset        => reset,
                  digit_input   => digit_input,
                  btn_enter     => btn_enter,
                  btn_next      => btn_next,
                  btn_program   => btn_program,
                  btn_override  => btn_override,
                  led_correct    => led_correct,
                  led_wrong     => led_wrong,
                  led_program_mode => led_program_mode,
                  led_locked    => led_locked,
                  seg           => seg,
                  an            => an);

    clk_process : process
```

```

begin
    clk <= '0';
    wait for TbPeriod / 2;
    clk <= '1';
    wait for TbPeriod / 2;
end process;

stim_proc: process
begin
    -- 1. Reset-----
    reset <= '1';
    digit_input <= (others => '0');
    btn_enter <= '0';
    btn_next <= '0';
    btn_program <= '0';
    btn_override <= '0';
    wait for 100 ns;

    reset <= '0';
    wait for 100 ns;

    -- 2. Override-----
    btn_override <= '1';
    wait for 50 ns;
    btn_override <= '0';

    wait for 200 ns;

    btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';
    wait for 100 ns;

    -- 3. Basic doorlock + autolock-----
    btn_next <= '1'; wait for 20 ns; btn_next <= '0';
    wait for 50 ns;

    digit_input <= x"1234";
    wait for 20 ns;
    btn_next <= '1'; wait for 20 ns; btn_next <= '0';
    wait for 50 ns;

    digit_input <= x"5678";
    wait for 20 ns;
    btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';

    wait for 2000 ns;

    -- Password salah -----
    btn_next <= '1'; wait for 20 ns; btn_next <= '0';
    wait for 50 ns;

    digit_input <= x"5678";
    wait for 20 ns;
    btn_next <= '1'; wait for 20 ns; btn_next <= '0';
    wait for 50 ns;

    digit_input <= x"1234";
    wait for 20 ns;
    btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';

    wait for 300 ns;

    btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';
    wait for 100 ns;

```

```

-- 4. Ganti Password 4444 5555 -----
btn_program <= '1'; wait for 20 ns; btn_program <= '0';
wait for 50 ns;

digit_input <= x"4444";
wait for 20 ns;
btn_next <= '1'; wait for 20 ns; btn_next <= '0';
wait for 50 ns;

digit_input <= x"5555";
wait for 20 ns;
btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';
wait for 100 ns;

-- 5. Verifikasi Password Baru-----
btn_next <= '1'; wait for 20 ns; btn_next <= '0';
wait for 50 ns;

digit_input <= x"4444";
wait for 20 ns;
btn_next <= '1'; wait for 20 ns; btn_next <= '0';
wait for 50 ns;

digit_input <= x"5555";
wait for 20 ns;
btn_enter <= '1'; wait for 20 ns; btn_enter <= '0';

wait for 200 ns;

wait;
end process;

end tb;

```

constraints.xdc

```

## Clock signal (100 MHz)
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { clk }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk}];

## switch untuk input (SW15-SW0)
set_property -dict { PACKAGE_PIN V10 IOSTANDARD LVCMOS33 } [get_ports { digit_input[15] }];
set_property -dict { PACKAGE_PIN U11 IOSTANDARD LVCMOS33 } [get_ports { digit_input[14] }];
set_property -dict { PACKAGE_PIN U12 IOSTANDARD LVCMOS33 } [get_ports { digit_input[13] }];
set_property -dict { PACKAGE_PIN H6 IOSTANDARD LVCMOS33 } [get_ports { digit_input[12] }];
set_property -dict { PACKAGE_PIN T13 IOSTANDARD LVCMOS33 } [get_ports { digit_input[11] }];
set_property -dict { PACKAGE_PIN R16 IOSTANDARD LVCMOS33 } [get_ports { digit_input[10] }];
set_property -dict { PACKAGE_PIN U8 IOSTANDARD LVCMOS33 } [get_ports { digit_input[9] }];
set_property -dict { PACKAGE_PIN T8 IOSTANDARD LVCMOS33 } [get_ports { digit_input[8] }];
set_property -dict { PACKAGE_PIN R13 IOSTANDARD LVCMOS33 } [get_ports { digit_input[7] }];
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports { digit_input[6] }];
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { digit_input[5] }];
set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports { digit_input[4] }];
set_property -dict { PACKAGE_PIN R15 IOSTANDARD LVCMOS33 } [get_ports { digit_input[3] }];
set_property -dict { PACKAGE_PIN M13 IOSTANDARD LVCMOS33 } [get_ports { digit_input[2] }];
set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports { digit_input[1] }];
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports { digit_input[0] }];

## Push Buttons
set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports { reset }];
set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports { btn_enter }];

```

```

set_property -dict { PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports { btn_next }];
set_property -dict { PACKAGE_PIN P17 IOSTANDARD LVCMOS33 } [get_ports { btn_program }];
set_property -dict { PACKAGE_PIN M17 IOSTANDARD LVCMOS33 } [get_ports { btn_override }];

## LEDs
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { led_locked }];
set_property -dict { PACKAGE_PIN V11 IOSTANDARD LVCMOS33 } [get_ports { led_program_mode
}];
set_property -dict { PACKAGE_PIN M16 IOSTANDARD LVCMOS33 } [get_ports { led_correct }];
set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led_wrong }];

## 7-Segment Display Cathodes
set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { seg[6] }];
set_property -dict { PACKAGE_PIN R10 IOSTANDARD LVCMOS33 } [get_ports { seg[5] }];
set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports { seg[4] }];
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports { seg[3] }];
set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports { seg[2] }];
set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports { seg[1] }];
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports { seg[0] }];

## 7-Segment Display Anodes
set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
set_property -dict { PACKAGE_PIN T9 IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { an[3] }];
set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports { an[4] }];
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { an[6] }];
set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { an[7] }];

set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

```

Lampiran 2. Video Presentasi

Link Google Drive:

https://drive.google.com/file/d/1IKeo8gwsk8P0_X0_JrViFICDjAMUY1h8/view?usp=sharing